

Relational-Indexed Retrieval-Augmented Generation (RI-RAG)

Laurie Dieudonné MAVOUNGOU

JK AI

lmavoungou@outlook.be

ABSTRACT

Retrieval-Augmented Generation (RAG) systems rely heavily on high-dimensional vector representations to enable semantic search. However, vector-based retrieval architectures may store substantial payload information and metadata alongside vector representations, motivating investigation into alternative storage organizations. This paper introduces *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG), a hybrid architecture that integrates relational database management systems (RDBMS) with vector stores to separate semantic indexing from authoritative relational storage. RI-RAG maps lightweight semantic representations to structured relational records through record identifiers, allowing the vector layer to serve as the semantic retrieval entry point while the relational database remains the authoritative source for complete content and structured metadata.

The prototype was evaluated through three experimental rounds. The first provided an initial comparison between Standard RAG and RI-RAG. The second refined the storage and resource evaluation using independent operating-system processes and additionally monitored the Windows MySQL server process. For the evaluated 13-record corpus, RI-RAG reduced the measured materialized Chroma representation from 63,954 bytes to 48,160 bytes, corresponding to a 24.70% reduction. The Python retrieval process exhibited a reduction in peak RSS increase from 29.20 MB for Standard RAG to 20.15 MB for RI-RAG, corresponding to a 31.01% lower Python-process peak RSS increase. When the evaluated Python and MySQL processes were measured together using simultaneous RSS samples, the combined peak RSS increase decreased from 29.20 MB to 25.51 MB, corresponding to a 12.65% reduction. The third round evaluated retrieval correspondence, evidence grounding, and the configured database-access boundary. Across ten ground-truth queries, RI-RAG achieved 90% top-1 retrieval accuracy on the evaluated query set, with a precision of 0.9000 and an F1 score of 0.9474.

These results provide empirical evidence that the RI-RAG storage organization can reduce measured memory consumption at the vector and process levels while introducing additional retrieval latency through relational record hydration. The combined memory result accounts for the monitored MySQL process required by RI-RAG, but does not represent every component of a complete deployment. The evaluation is also limited by the small 13-record corpus and limited repeated trials. The results therefore establish an empirical prototype result rather than evidence of large-scale enterprise scalability.

KEYWORDS

Relational database management systems, Information Retrieval, RAG, Vector Databases, Memory Optimization

1 OVERVIEW

Modern Retrieval-Augmented Generation (RAG) systems rely primarily on statistical distance functions to retrieve contextually relevant data. However, semantic similarity retrieval alone does not necessarily provide an explicit relational source of truth for the retrieved record. To address this limitation, we present a hybrid retrieval architecture in which candidate vector references are mapped to corresponding records in a relational database. This additional retrieval stage separates semantic candidate selection from authoritative record retrieval.

By isolating full unstructured payloads within the relational layer and maintaining semantic representations and record identifiers in the vector layer, RI-RAG provides an alternative storage organization whose resource behavior can be measured experimentally. The central hypothesis investigated in this work is that separating large authoritative payloads from the vector retrieval layer can reduce the memory footprint associated with the retrieval process and the amount of data materialized by the vector store.

The refined isolated-process benchmark provides empirical evidence for this hypothesis on the evaluated prototype. The Python retrieval process exhibited a 31.01% lower peak RSS increase than the Standard RAG configuration, decreasing the measured peak increase from 29.20 MB to 20.15 MB. When the Windows MySQL server process was also monitored, the combined peak RSS increase decreased from 29.20 MB for Standard RAG to 25.51 MB for RI-RAG, corresponding to a 12.65% reduction. The measured materialized Chroma representation was also reduced by 24.70%, from 63,954 bytes to 48,160 bytes.

These measurements refer to different levels of the evaluated architecture. The 24.70% result describes the materialized vector-store representation, the 31.01% result describes the Python retrieval process, and the 12.65% result describes the combined peak RSS increase of the evaluated Python and MySQL processes. The combined result should not be interpreted as a 12.65% reduction in total deployment memory because the benchmark does not include every operating-system, application, database, network, or infrastructure component.

The architecture introduces a retrieval-latency trade-off. Because RI-RAG resolves the retrieved vector identifier through MySQL before returning the authoritative record, the refined benchmark measured additional latency compared with the Standard RAG configuration. The current benchmark produced a mean retrieval latency of 135.81 ms for Standard RAG and 216.32 ms for RI-RAG. These measurements include relational hydration for RI-RAG and exhibit substantial variation, so they should be considered current preliminary measurements pending warm-up procedures and repeated controlled trials before stronger latency conclusions are drawn.

The relational layer can also provide a boundary at which database access-control and integrity mechanisms may be applied. These

properties depend on the database and application configuration; the experiments in this report evaluate only the configured database-access behavior rather than comprehensive enterprise security.

Section 2 discusses related work, and Section 3 describes the research problem and proposed solution. Section 4 presents the dataset and prototyping scope. The empirical benchmarks and performance analysis are presented in Section 5. Section 7 presents foundational concepts and explains how RI-RAG addresses the investigated problems. Section 8 presents broader impact and target applications, while Section 9 refers to the GitHub repository containing the implementation. The conclusion and future work are presented in Section 10.

2 RELATED WORK

Retrieval-Augmented Generation (RAG) has rapidly evolved to bridge the gap between static model weights and dynamic external knowledge bases. Existing implementations can involve challenges related to memory organization, structural grounding, and access control. In this section, we discuss prior approaches across these dimensions and describe the design distinction of *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG).

2.1 Memory Optimization and Hardware Offloading

As context windows expand, inference costs can increase due to high-dimensional memory consumption. Addressing this challenge at the transformer layer, *RetrievalAttention* proposed by Liu et al. [3] offloads key-value (KV) caches from GPU to CPU memory, relying on dynamic vector search to recover approximate nearest neighbors during generation.

While Liu et al. [3] and our work address memory organization at different levels of the system, they operate at fundamentally different architectural tiers. *RetrievalAttention* operates on internal model activations and memory tiers, whereas RI-RAG operates at the application and database layers. RI-RAG separates full text payloads from vector-search information by storing complete authoritative records in a relational database while maintaining semantic representations and record identifiers in the vector store. This changes the organization of information retrieval and storage at the application/database layer without modifying the model's attention mechanisms or internal architecture.

The present work therefore investigates memory reduction at the retrieval-storage layer rather than memory optimization within model inference. The refined resource evaluation provides empirical measurements of this architectural effect at the prototype process and vector-store levels.

2.2 Hierarchical and Structured Retrieval Paradigms

To provide more structured retrieval, recent literature has explored graph-based and hierarchical indexing approaches. Edge et al. [1] introduced GraphRAG, which extracts entity-relationship graphs to enable hierarchical community summaries. Similarly, Gutierrez et al. [2] proposed HippoRAG, a neurobiologically inspired long-term memory framework that models associative retrieval through personalized PageRank over knowledge graphs.

In contrast to graph-based approaches, RI-RAG uses relational database management systems (RDBMS) to connect semantic retrieval results with structured authoritative records through record identifiers and relational schemas. The architecture therefore provides a relational resolution step without requiring graph extraction or graph traversal.

2.3 Enterprise Security and Access Boundary Constraints

Vector stores and graph-based retrieval systems can use different approaches to access control and filtering. RI-RAG instead places the authoritative record in a relational layer, creating a database boundary at which access-control policies can potentially be applied.

RI-RAG decouples the vector representation from the underlying enterprise record. By routing retrieved candidates through a relational layer, the architecture provides a point at which database-level privileges and application-level access-control policies can be applied before relational content is returned to the generation component. The effectiveness of such controls depends on the deployment configuration.

3 RESEARCH PROBLEM

Existing RAG implementations present three architectural considerations addressed by this work:

- (1) Semantic retrieval alone does not necessarily expose an explicit relationship between a retrieved vector representation and an authoritative structured record.
- (2) Vector indexes may contain payloads and metadata in addition to embeddings. Separating full payload storage from semantic indexing provides an alternative storage organization whose resource behavior can be measured experimentally.
- (3) Enterprise deployments require access-control and audit mechanisms around retrieved information. RI-RAG places authoritative records in a relational database where such mechanisms can be configured, although the present report does not provide a comprehensive security evaluation.

The primary resource hypothesis investigated by this work is that moving complete authoritative payloads out of the vector retrieval layer can reduce the memory footprint associated with the evaluated retrieval process and reduce the amount of materialized payload information maintained by the vector store.

3.1 Contributions

The contributions of this work are:

- Introduction of the RI-RAG hybrid relational-vector retrieval architecture.
- Prototype implementation using semantic summaries in a vector store and authoritative records in MySQL.
- Experimental comparison of the materialized vector-store representation between Standard RAG and RI-RAG.
- Isolated-process evaluation of peak RSS increase to investigate the memory footprint of the two retrieval configurations.

- Extended memory evaluation that additionally monitors the MySQL server process required by RI-RAG.
- Experimental characterization of the retrieval-latency trade-off introduced by relational record hydration.
- Evaluation of retrieval accuracy using ground-truth query-to-record mappings.
- Evaluation of evidence grounding through primary-key resolution and authoritative record hydration.
- Evaluation of the configured database-access boundary.
- Identification of limitations and future experiments required for larger-scale evaluation.

3.2 Technical Methodology

RI-RAG separates semantic indexing from full document storage by using two complementary storage layers. The vector database acts as the semantic entry point for retrieval, while the relational database acts as the authoritative source of truth for complete document content and structured metadata.

The semantic summary of each document is used to generate its vector embedding. Instead of storing the complete document payload within the vector index, RI-RAG maintains a relationship between the vector representation and the corresponding relational record through its record identifier. The vector store therefore provides semantic retrieval, while the relational database provides record hydration and authoritative information.

The major architectural components are:

- **Relational verification layer:** Maintains the authoritative document content and structured metadata.
- **Structured record resolution:** Uses the retrieved record identifier to identify the corresponding relational record.
- **Payload separation:** Keeps full document payloads in the relational database while limiting the vector layer to semantic representations and required identifiers.

The retrieval workflow consists of the following stages:

- (1) **Semantic entry point:** The user query is converted into an embedding and compared with the vector representations stored in the vector database.
- (2) **Relational hydration:** The record identifier returned by vector retrieval is used to retrieve the corresponding authoritative record from MySQL.
- (3) **Entry-only storage:** The vector layer maintains the semantic representation and record identifier rather than the complete authoritative payload.
- (4) **Ingestion and synchronization:** The system maintains the relationship between vector entries and relational records during ingestion and updates.
- (5) **Hydration and ranking engine:** Retrieved identifiers are resolved into relational records for subsequent processing.
- (6) **Access control and filtering:** Relational database controls and application-level filtering can be applied to authoritative records after retrieval.

The RI-RAG retrieval workflow can be summarized as presented in Figure 1.

3.3 Vector Database Sample

The vector database contains the lightweight semantic representation and the relationship needed to retrieve the authoritative record. The vector database does not store the complete authoritative document content.

```
{
  "id": <primary_key>,
  "metadata": {
    "file_name": <file_name>,
    "file_id": <primary_key>
  },
  "documents": <semantic_summary>,
  "embedding": <embedding>
}
```

3.4 MySQL Table Schema

MySQL serves as the authoritative source of truth for the complete document payload and structured metadata.

```
CREATE TABLE file (
  file_id VARCHAR(50) NOT NULL,
  file_name VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  semantic_summary TEXT NOT NULL,
  grade FLOAT NOT NULL,
  comments TEXT NOT NULL,
  CONSTRAINT file_pk PRIMARY KEY (file_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

The full content remains available in the relational layer and can be retrieved through the relationship established between the vector record identifier and the corresponding relational record.

3.5 Experimental Description

Table 1 summarizes the experimental rounds conducted in this study, their objectives, and the corresponding implementation.

The experiments intentionally distinguish among three related but different measurements:

- **Materialized vector-store representation:** the measured size of the Python objects materialized from the Chroma collection by the benchmark.
- **Process-level memory consumption:** the increase in resident set size (RSS) from the measured process baseline to its observed peak during the isolated benchmark.
- **Combined process memory consumption:** the simultaneous RSS of the evaluated Python process and Windows MySQL server process.
- **Retrieval latency:** the elapsed time required to perform retrieval under the respective architecture.

The process-level RSS measurement includes the memory behavior of the evaluated Python process, including the runtime, embedding model, Chroma components, and operations executed by the benchmark. The extended benchmark additionally monitors the Windows mysqld.exe process. The combined measurement is

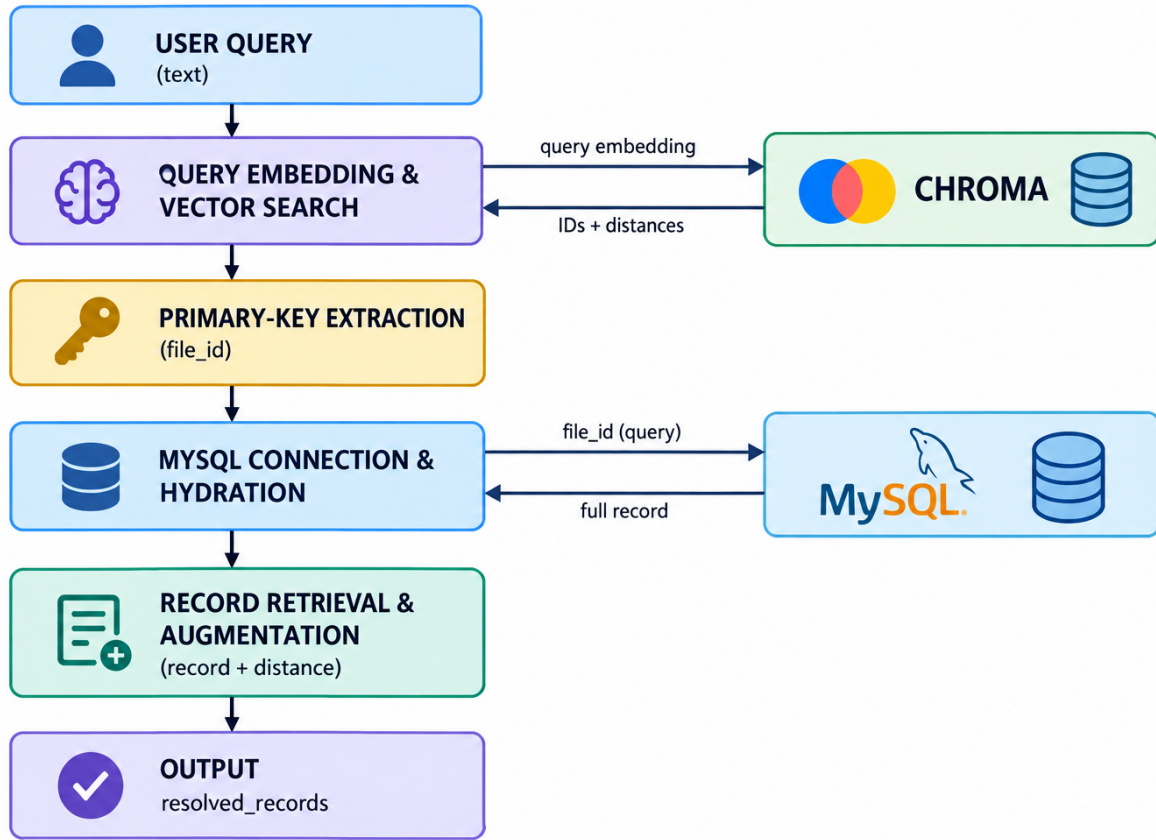


Figure 1: RI-RAG retrieval workflow.

obtained from simultaneous RSS samples rather than by adding independently observed peak values.

4 DATASET AND PROTOTYPING SCOPE

To validate the RI-RAG architectural prototype under controlled conditions, we constructed a synthetic benchmark dataset comprising 13 enterprise and security-focused records (DOC-001 through DOC-013). The dataset is intentionally small and serves to demonstrate the relational-vector decoupling and retrieval workflow rather than to establish large-scale production behavior.

The dataset is modeled within a relational schema enforced via MySQL:

- **Controlled Payload Expansion:** The text payloads (content) were expanded through controlled repetition to create a reproducible test environment for examining storage behavior. This provides a controlled payload-expansion experiment rather than a representation of naturally occurring enterprise documents.
- **Domain Relevance:** The records span enterprise security and architecture topics, including consistency guidelines, autonomous agent safety, vector optimization, role-based

access control, audit logging, adversarial robustness, and federated privacy.

- **Structured Attributes:** Each record contains a primary key (file_id), file name, full text content, semantic summary, a performance grade (FLOAT), and operational comments.

To implement and evaluate the RI-RAG prototype, specific system configurations and parameters were established across both the relational and vector storage layers:

- **Dataset Size:** The evaluation corpus consists of 13 synthetic enterprise and security-focused records (DOC-001 through DOC-013) stored in a MySQL relational database.
- **Payload Scale:** Text payloads (content) were expanded through controlled repetition, averaging approximately 750 to 1,200 characters per document while vector-side semantic summaries remained substantially shorter.
- **Embedding Model:** Semantic representations are generated using the sentence-transformers/all-MiniLM-L6-v2 model.
- **Embedding Dimension:** The vector representation uses an embedding dimension of 384.
- **Indexing Strategy:** RI-RAG generates embeddings exclusively from the relational semantic_summary attribute. The

Table 1: RI-RAG experimental evaluation.

Experiment	Objective	Implementation and Measurement
Round 1: Initial Benchmark	Compare Standard RAG and RI-RAG at the vector-store and retrieval levels.	Both systems were populated from the same MySQL corpus. The experiment measured the materialized representation returned by the Chroma collection and externally measured retrieval latency. The initial measurement produced 8,872 B for Standard RAG and 8,273 B for RI-RAG, with corresponding mean retrieval times of 7.31 ms and 22.49 ms.
Round 2: Refined Resource Evaluation	Measure vector-store representation, process-level memory consumption, and retrieval latency using independent processes while accounting for the MySQL dependency.	Each configuration was executed in an independent operating-system process. Both systems loaded the same 13-record corpus. Python RSS and the Windows MySQL server RSS were monitored. Standard RAG produced a 29.20 MB Python peak RSS increase, while RI-RAG produced a 20.15 MB Python peak RSS increase. The combined Python + MySQL peak RSS increase was 29.20 MB for Standard RAG and 25.51 MB for RI-RAG. The materialized Chroma representations were 63,954 bytes and 48,160 bytes respectively.
Round 3: Retrieval and Functional Evaluation	Evaluate retrieval correspondence, evidence grounding, and the configured database-access boundary.	Ten ground-truth queries were executed through the live HybridRIRAGSystem using top-1 retrieval. The expected record identifiers were compared with the retrieved identifiers. Evidence grounding was tested using DOC-001 and authoritative MySQL hydration. Database-boundary behavior was tested using the configured <code>test_user</code> account and <code>CURRENT_USER()</code> .

complete document content remains isolated within MySQL and is retrieved through primary-key resolution.

- **Chroma Configuration:** The vector retrieval layer utilizes a local `chromadb.Client()` instance. RI-RAG stores the semantic summary as the Chroma document together with lightweight metadata containing `file_id` and `file_name`.

For the Standard RAG comparison, the vector store contains the complete content payload together with additional metadata. Consequently, the two configurations differ primarily in where the authoritative payload is stored.

The refined resource benchmark used independent processes for the two configurations. The Standard RAG and RI-RAG processes were therefore initialized separately, allowing baseline and peak RSS measurements to be obtained without simultaneously running both retrieval implementations in the same process.

The extended benchmark additionally monitored the Windows MySQL server process. This is important because RI-RAG depends on MySQL for authoritative record hydration, whereas the Standard RAG baseline does not require the same relational retrieval step during query resolution.

Due to the limited corpus size and controlled payload construction, this dataset is not sufficient to establish large-scale enterprise scaling behavior. Larger corpora, more diverse documents, substantially larger payload sizes, repeated trials, and full deployment-level resource measurements remain necessary for future evaluation.

5 EMPIRICAL BENCHMARKS & PERFORMANCE ANALYSIS

5.1 Round 1: Initial Benchmark

The first evaluation compares a Standard RAG configuration with the RI-RAG prototype using vector-store representation size and

retrieval latency. The purpose of this benchmark is to measure the initial effect of separating semantic indexing from authoritative relational storage.

Table 2: Initial comparison between Standard RAG and RI-RAG.

Metric	Standard RAG	RI-RAG	Difference
Vector-store representation	8,872 B	8,273 B	599 B
Representation reduction	–	–	6.8%
Query latency	7.31 ms	22.49 ms	15.18 ms

The initial benchmark measured a 599-byte difference between the two vector-store configurations, corresponding to a 6.8% reduction in the measured materialized vector-store representation for RI-RAG. This measurement applies only to the vector-store representation returned by the benchmark and does not represent total application or system memory consumption.

The latency results show the trade-off introduced by the additional relational hydration step. Standard RAG achieved an average retrieval latency of approximately 7.31 ms, while RI-RAG measured approximately 22.49 ms under the initial test conditions.

Round 1 is retained as the historical initial benchmark. The refined Round 2 experiment uses a more comprehensive resource methodology and therefore provides the primary current memory characterization used in this report.

5.2 Round 2: Refined Storage, Memory, and Retrieval Performance

The second evaluation refined the experimental methodology by executing Standard RAG and RI-RAG in independent operating-system processes and monitoring the Windows MySQL server process separately.

The same 13-record corpus was used for both configurations. Standard RAG stores the complete content payload in the vector database, whereas RI-RAG stores the semantic summary and lightweight identifiers while retrieving authoritative content from MySQL.

The refined benchmark provides three distinct memory-related measurements.

First, the measured materialized Chroma representation decreased from 63,954 bytes for Standard RAG to 48,160 bytes for RI-RAG. The corresponding reduction is:

$$\frac{63,954 - 48,160}{63,954} \times 100 \approx 24.70\%. \quad (1)$$

This result demonstrates that the RI-RAG configuration maintained a smaller measured vector-store representation for the evaluated corpus.

Second, the isolated Python process benchmark measured the increase in resident set size (RSS) from the process baseline to its observed peak. Standard RAG increased from a baseline of 565.35 MB to a peak of 594.55 MB, producing a peak increase of 29.20 MB. RI-RAG increased from 565.84 MB to 585.98 MB, producing a peak increase of 20.15 MB.

The relative reduction in Python peak RSS increase is:

$$\frac{29.20 - 20.15}{29.20} \times 100 \approx 31.01\%. \quad (2)$$

Thus, under the evaluated 13-record configuration, RI-RAG exhibited a **31.01% lower Python-process peak RSS increase** than Standard RAG.

Third, the extended benchmark monitored the MySQL server process required by RI-RAG. Standard RAG exhibited a measured MySQL peak RSS increase of 0.00 MB, while RI-RAG exhibited a 5.76 MB increase. This additional resource consumption is expected because RI-RAG uses MySQL for authoritative record hydration.

To account for this architectural dependency, the benchmark calculated combined RSS using simultaneous Python and MySQL RSS samples:

$$RSS_{\text{combined}}(t) = RSS_{\text{Python}}(t) + RSS_{\text{MySQL}}(t). \quad (3)$$

The Standard RAG configuration exhibited a combined peak RSS increase of 29.20 MB, while RI-RAG exhibited a combined peak RSS increase of 25.51 MB.

The relative reduction is:

$$\frac{29.20 - 25.51}{29.20} \times 100 \approx 12.65\%. \quad (4)$$

Thus, after accounting for the monitored MySQL server process, RI-RAG exhibited a **12.65% lower combined peak RSS increase** than Standard RAG under the evaluated workload.

This 12.65% result is the primary end-to-end process-level memory result reported by the updated benchmark. It is more conservative than the 31.01% Python-only measurement because it accounts for the additional database process required by RI-RAG.

The combined peak is measured directly from simultaneous samples. The benchmark therefore does not add the independently observed Python and MySQL peak values, because those peaks may occur at different points in time.

The memory results can consequently be summarized at three architectural levels:

These measurements demonstrate that RI-RAG reduced measured memory consumption under the evaluated workload. They do not establish that every possible RI-RAG deployment will consume 12.65% less total infrastructure memory.

The benchmark also measured retrieval latency. The current isolated-process benchmark produced the following measurements:

The RI-RAG latency includes the additional relational database lookup required to resolve authoritative records. The current measurements exhibit substantial variation, particularly in the Standard RAG baseline. They should therefore be treated as preliminary current measurements rather than as a definitive latency characterization. Warm-up procedures, repeated independent trials, controlled database state, and distributional analysis are required before stronger conclusions about latency performance are drawn.

The current benchmark confirms the retrieval identifiers for three direct comparison queries:

- DOC-002 for the agent-failure and drift query;
- DOC-005 for the role-based access-control query;
- DOC-004 for the vector-index memory-reduction query.

Both configurations returned the same identifiers for these three queries.

RI-RAG also supports batched retrieval. A batch containing three queries completed in 435.73 ms in the current benchmark. This demonstrates support for multi-query retrieval but does not establish throughput scalability.

Overall, Round 2 provides the principal empirical evidence for the resource behavior investigated by this paper: RI-RAG reduced the measured vector-store representation by 24.70%, reduced Python-process peak RSS increase by 31.01%, and reduced the combined peak RSS increase of the evaluated Python and MySQL processes by 12.65%.

5.3 Round 3: Retrieval Accuracy, Evidence Grounding, and Database Boundary

The third evaluation examined whether the RI-RAG retrieval pipeline could resolve ground-truth records, hydrate authoritative content, and operate through the configured database-access boundary.

Ten queries were constructed with explicit expected document identifiers ranging from DOC-001 to DOC-010. Each query was submitted to the actual HybridRIRAGSystem.retrieve() implementation using top-1 retrieval.

The retrieval evaluation produced nine correct top-1 matches out of ten queries, corresponding to 90.0% top-1 retrieval accuracy on the evaluated ten-query ground-truth set. The resulting precision was 0.9000 and the F1 score was 0.9474 under the implemented evaluation formulation.

Table 3: Round 2 isolated-process and combined-process resource results.

Metric	Standard RAG	RI-RAG
Baseline Python RSS	565.35 MB	565.84 MB
Peak Python RSS	594.55 MB	585.98 MB
Python peak RSS increase	29.20 MB	20.15 MB
Baseline MySQL RSS	48.11 MB	48.26 MB
Peak MySQL RSS	48.11 MB	54.02 MB
MySQL peak RSS increase	0.00 MB	5.76 MB
Baseline combined RSS	613.46 MB	614.09 MB
Peak combined RSS	642.67 MB	639.60 MB
Combined peak RSS increase	29.20 MB	25.51 MB
Materialized Chroma representation	63,954 B	48,160 B

Table 4: Memory results at different measurement levels.

Measurement level	Reduction	Interpretation
Chroma representation	24.70%	Smaller measured vector-store representation
Python process	31.01%	Lower peak RSS increase of the retrieval process
Python + MySQL	12.65%	Lower combined peak RSS increase of evaluated processes

Table 5: Round 2 retrieval latency measurements.

Metric	Standard RAG	RI-RAG
Mean latency	135.81 ms	216.32 ms
Minimum latency	5.49 ms	142.91 ms
Maximum latency	237.63 ms	293.05 ms
3-query batch	–	435.73 ms

Table 6: Round 3 functional evaluation results.

Metric	RI-RAG Result
Ground-truth queries	10
Correct top-1 retrievals	9 / 10
Top-1 retrieval accuracy	90.0%
Precision	0.9000
F1 score	0.9474
Known mismatch	DOC-003 → DOC-009
Evidence grounding	PASS
Database boundary	Verified under test_user

The only mismatch occurred for the query targeting DOC-003. The expected record was DOC-003, while the system retrieved DOC-009. The result indicates semantic overlap in the embedding space between these technically related concepts. It also illustrates that the relational architecture does not by itself guarantee semantic retrieval correctness; retrieval quality remains dependent on the semantic representation used for vector search.

For the evidence-grounding experiment, a query targeting DOC-001 was submitted through the RI-RAG retrieval pipeline. The returned

record identifier was resolved through the database and the authoritative content field was inspected. The expected core sentence was present in the retrieved authoritative record, resulting in a successful grounding test.

The database-boundary experiment connected to MySQL using the configured test_user account and inspected CURRENT_USER(). The experiment returned test_user@localhost, verifying that the evaluated database operation was executed under the configured account. This demonstrates the configured database-access boundary for the tested operation. It does not establish comprehensive enterprise RBAC effectiveness, privilege-escalation resistance, or a complete security evaluation.

6 DISCUSSION

The refined benchmark measured the materialized Chroma representation at 63,954 bytes for Standard RAG and 48,160 bytes for RI-RAG, corresponding to a 24.70% reduction under the evaluated 13-record configuration.

At the Python-process level, Standard RAG exhibited a peak RSS increase of 29.20 MB, while RI-RAG exhibited a peak RSS increase of 20.15 MB. This corresponds to a 31.01% reduction in the peak RSS increase:

$$\frac{29.20 - 20.15}{29.20} \times 100 \approx 31.01\%. \quad (5)$$

Because RI-RAG requires MySQL for authoritative record hydration, the extended benchmark also monitored the Windows MySQL server process. When simultaneous Python and MySQL RSS measurements were considered together, Standard RAG exhibited a combined peak RSS increase of 29.20 MB, while RI-RAG exhibited a combined peak RSS increase of 25.51 MB. This corresponds to a 12.65% reduction:

$$\frac{29.20 - 25.51}{29.20} \times 100 \approx 12.65\%. \quad (6)$$

The 12.65% result is the primary end-to-end process-level memory result of the updated benchmark because it accounts for the additional MySQL process introduced by RI-RAG. It should not be interpreted as a universal 12.65% reduction in total deployment memory. The measurement covers the evaluated Python and MySQL processes but does not include every operating-system, filesystem, networking, infrastructure, or external service component.

The memory reduction was accompanied by a measurable retrieval-latency trade-off. The current benchmark measured mean retrieval latencies of 135.81 ms for Standard RAG and 216.32 ms for RI-RAG. The additional latency is consistent with the relational hydration step required to resolve the retrieved record identifier into authoritative MySQL content. However, the current latency measurements exhibit substantial variation and should therefore be followed by warm-up procedures and repeated trials before stronger performance conclusions are made.

The direct three-query comparison returned the same top-1 identifiers in both systems: DOC-002, DOC-005, and DOC-004. Round 3 provided a broader functional evaluation and correctly retrieved nine of ten ground-truth records at top-1, corresponding to 90.0% top-1 retrieval accuracy on the evaluated ten-query set, with a precision of 0.9000 and an F1 score of 0.9474.

The single Round 3 mismatch, between DOC-003 and DOC-009, demonstrates that memory-footprint optimization does not independently determine semantic retrieval quality. Retrieval performance remains dependent on the semantic representations used by the vector layer and the embedding model.

The evidence-grounding experiment successfully resolved DOC-001 through the relational layer, demonstrating that the vector result can be connected to an authoritative database record. The database-boundary experiment also verified execution under the configured `test_user` account. These results demonstrate the implemented retrieval and database-resolution mechanisms but do not constitute a comprehensive security evaluation.

The dataset used is intentionally small and synthetic, and the controlled payload expansion does not establish behavior on naturally occurring large enterprise documents. The current results should therefore be understood as prototype evidence supporting the RI-RAG memory-footprint hypothesis rather than as proof of a universal resource reduction across all RAG deployments.

7 FOUNDATIONAL CONCEPTS & PROBLEM SOLVING

7.1 What is Memory and Why Does It Matter?

In computer science and software architecture, memory refers to the high-speed volatile storage space used by applications to hold active data during processing. Memory is a finite resource, and excessive memory requirements can increase infrastructure requirements and operational costs as AI systems scale. In RAG systems, the amount of data maintained and materialized by the retrieval layer is therefore an important resource consideration.

RI-RAG investigates whether separating semantic representations from authoritative payloads can reduce the memory footprint associated with the retrieval process and reduce the amount of payload information maintained by the vector layer. The refined experiment provides empirical evidence for this hypothesis in the evaluated prototype: RI-RAG exhibited a 31.01% lower peak RSS increase in the Python retrieval process and a 12.65% lower combined peak RSS increase when the monitored MySQL process was included. The measured materialized Chroma representation was also 24.70% smaller.

These measurements characterize different levels of the architecture rather than a single universal memory-reduction percentage.

The combined 12.65% reduction provides the most conservative process-level evidence of memory reduction, as it accounts for the additional MySQL component required by RI-RAG.

7.2 What is Traceability and Why Does It Matter?

Traceability is the ability to associate retrieved information with its source record and relevant metadata. In retrieval systems, this association can help engineers inspect the provenance of information supplied to the generation stage.

7.3 How RI-RAG Addresses the Investigated Problems

- **A. Supporting Traceability:** RI-RAG separates the semantic retrieval entry point from the authoritative database. A retrieved candidate can therefore be resolved to a specific relational record before its content is supplied to subsequent processing. This provides an explicit record-resolution mechanism that can support evidence-grounded generation.

```
# Vector search identifies the record
```

```
matched_id = vector_results["ids"][0][0]
```

```
# Retrieve authoritative record
```

```
record = self.db.fetch_by_id(matched_id)
```

- **B. Centralizing Metadata Updates:** Metadata and full content remain in the relational source of truth. Updates can therefore be performed at the relational layer without requiring duplication of the complete payload in the vector store.

```
-- Metadata update through SQL
```

```
UPDATE file
```

```
SET grade = 9.9
```

```
WHERE file_id = 'DOC-001';
```

- **C. Supporting Transactional Storage and Access Control:** MySQL provides transactional and privilege mechanisms that can be configured for authoritative records. Their effectiveness depends on database configuration and was not comprehensively evaluated in the present benchmark.
- **D. Supporting Debugging and Content Evolution:** Because authoritative content and metadata are stored relationally, engineers can inspect the corresponding record and its metadata when investigating retrieval or generation behavior. The impact on re-indexing requirements depends on which fields are modified.
- **E. Reduced Vector-Layer Memory Footprint:** RI-RAG removes the need for the vector layer to maintain the complete authoritative payload, retaining instead the semantic representation and lightweight identifiers required for retrieval. The refined benchmark provides empirical evidence of the resulting resource effect. The RI-RAG process exhibited a 31.01% lower peak RSS increase than Standard RAG, while the measured materialized Chroma representation was reduced by 24.70%. When the monitored MySQL process was included, the combined peak RSS increase was 12.65% lower.

The three results describe different levels of the architecture. The 24.70% measurement concerns the materialized vector-store representation, the 31.01% measurement concerns the Python retrieval process, and the 12.65% measurement concerns the combined Python and MySQL processes. Future experiments should therefore measure complete deployment resource consumption to determine the end-to-end infrastructure effect.

8 BROADER IMPACT & TARGET APPLICATIONS

The RI-RAG architecture may be applicable to systems where semantic retrieval and authoritative structured storage have different operational requirements.

- **Enterprise Cybersecurity Operations:** RI-RAG could be used to connect semantic retrieval of security alerts with authoritative incident records, historical evidence, or operational playbooks. Database and application-level access policies could be applied to the authoritative records.
- **Enterprise Knowledge Systems:** Systems containing large authoritative records may use the architecture to maintain semantic retrieval information separately from complete relational payloads. The prototype results demonstrate measurable reductions in vector representation and process-level memory under the evaluated conditions. The resulting resource behavior would depend on payload size, record count, database configuration, and workload characteristics.
- **Cloud and Infrastructure Applications:** The architecture could allow some deployments to maintain large authoritative payloads in relational storage rather than duplicating them within the vector layer. The observed 12.65% reduction in combined peak RSS increase provides initial prototype evidence for this resource-management approach. The economic implications remain workload- and infrastructure-specific and require evaluation of complete deployment costs, including database resources.

9 SOURCE CODE AND IMPLEMENTATION

The implementation details, database schema scripts, synchronization pipelines, and benchmark implementations for the Relational-Indexed Retrieval-Augmented Generation (RI-RAG) architecture are made publicly available to support reproducibility. The complete source code and architectural modules can be accessed on GitHub:

<https://github.com/lmav-jpeg/RI-RAG>

The repository also provides the implementation used to reproduce the relational-vector retrieval workflow and the benchmark configurations described in this report.

Contributions, issue tracking, and additional benchmarks are welcomed through the repository.

10 CONCLUSION & FUTURE WORK

This work introduced Relational-Indexed Retrieval-Augmented Generation (RI-RAG), a hybrid retrieval architecture that separates

semantic indexing from authoritative relational storage. The prototype uses a vector database as the semantic retrieval layer while maintaining MySQL as the authoritative source of truth for complete document content and structured metadata. Retrieved record identifiers are resolved through the relational database before authoritative content is returned.

The experimental results provide empirical evidence for the memory-footprint reduction targeted by the architecture. Future work should evaluate substantially larger corpora and payloads while repeating the experiments across multiple independent runs. Record count and payload size should be varied independently to determine how the memory-footprint difference changes as the corpus grows. Confidence intervals and statistical tests should also be incorporated into repeated latency and memory measurements.

Future evaluations should additionally measure:

- total vector-store storage on disk;
- complete MySQL storage requirements;
- MySQL server RSS and buffer-pool consumption;
- total end-to-end deployment memory;
- retrieval throughput under concurrent workloads;
- latency distributions across repeated trials;
- retrieval accuracy on larger and more diverse datasets;
- the effect of different semantic-summary lengths;
- the effect of different payload sizes;
- synchronization and update costs between relational and vector layers.

A further objective is to determine whether the resource difference observed in the prototype becomes more pronounced as document payloads increase. Such experiments would provide stronger evidence regarding the central hypothesis of RI-RAG: that semantic retrieval can remain effective while large authoritative payloads are maintained outside the vector layer.

11 AI USAGE ACKNOWLEDGEMENT

This work was gratefully inspired by concepts explored in “Vector Databases Demystified” by Abhinav Verma (Published on Medium). Artificial intelligence tools were utilized in the preparation, code implementation, and generation of this report. Specifically, AI models were prompted and consulted to design and test implementation scripts, evaluate architecture benchmarks, and synthesize and structure the content of this document. AI tools were also used for English language editing, formatting assistance, and LaTeX code structuring. All core concepts, system design, experimental direction, and conclusions remain the original work of the author(s).

REFERENCES

- [1] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. (April 2024). <https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/>
- [2] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2025. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. arXiv:2405.14831 [cs.CL] <https://arxiv.org/abs/2405.14831>
- [3] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In *NeurIPS 2025*. <https://www.microsoft.com/en-us/research/publication/retrievalattention-accelerating-long-context-llm-inference-via-vector-retrieval/>